

UNIT 2 :

PROCESS MANAGEMENT

[14-16 marks]

Hamid Ali

Lecturer, Computer Engg.

A.I- ARKP, Panvel

Topics to be covered...

2	TLO 2.1 Explain the different states of a process. TLO 2.2 Describe the functions of different component of process stack in PCB (Process Control Block). TLO 2.3 Explain multiple processes access shared resources without interfering each other. TLO 2.4 Compare Multithreading models.	Unit - II Process Management 2.1 Processes: process state, process control block 2.2 Process Scheduling: scheduling queues, types of schedulers, context switch 2.3 Inter Process Communication: Shared memory system, Message passing system 2.4 Threads: Benefits, User and Kernel level threads, Multithreading Models: One to One, Many to One, Many to Many 2.5 Execute process commands like: top, ps, kill, wait, sleep, exit, nice
---	---	---

Process - Concept

- A program is a set of instructions. Program under execution is known as a Process.
- A program by itself is not a process, a program is a passive entity whereas a process is an active entity.

Passive Entity → It is a set of instructions stored on a disk. It is not running.

Active Entity → It is a program that is currently running/executing in memory.

- A program becomes a process when an executable file is loaded into the memory.

Process - Concept

- A process in memory is divided into several distinct sections, each serving a different purpose. Here's how a process typically looks in memory:
- **Text Section:** A Process, sometimes known as the Text Section, also includes the current activity represented by the value of the [Program Counter](#).
- **Stack:** The stack contains temporary data, such as function parameters, returns addresses, and local variables.
- **Data Section:** Contains the global variable.
- **Heap Section:** [Dynamically memory allocated](#) to process during its run time.

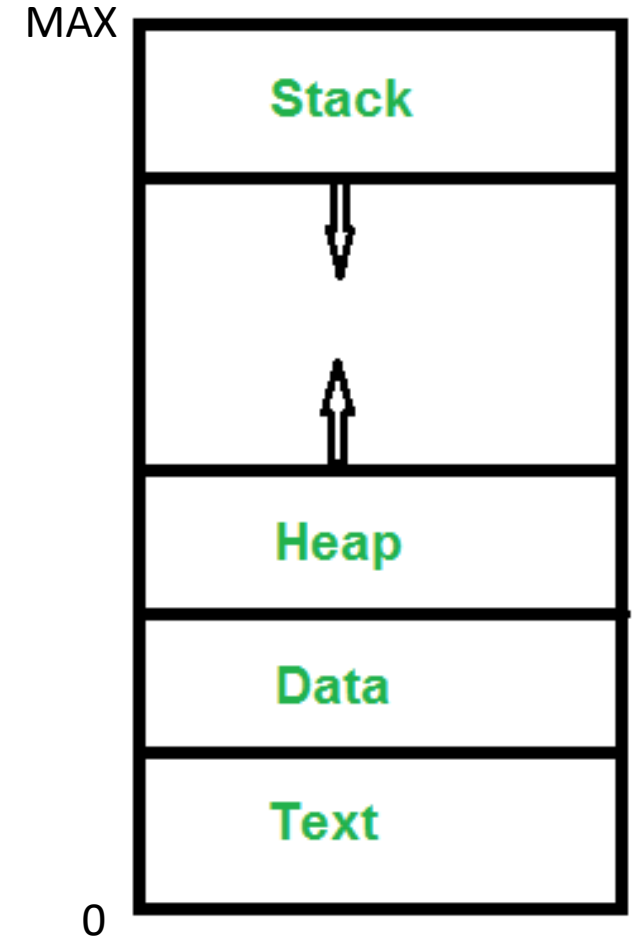


Fig: How process looks in memory

Process State Transition (MMIMP)

A process in an operating system passes through multiple states as it begins execution, waits for resources, gets scheduled, runs, and eventually finishes. These stages collectively describe the complete lifecycle of a process.

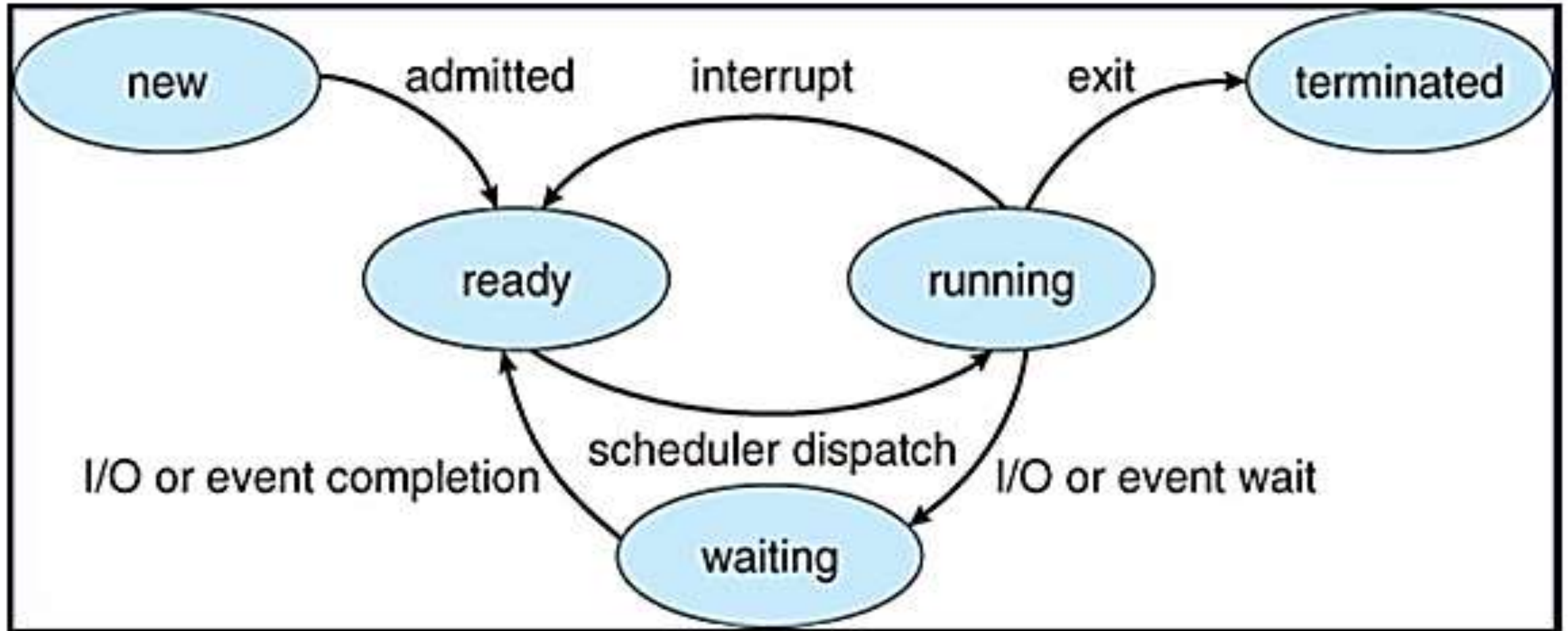


Fig: Process state diagram

Process State Transition (MMIMP)

- **New:** When a process enters into the system, it is in new state. In this state a process is created.
- **Ready:** When the process is loaded into the main memory, it is ready for execution. In this state the process is waiting for processor allocation.
- **Running:** When processor is available, system selects one process from main memory and executes all the instructions from that process. So, when a process is in execution, it is in running state. In single user system, only one process can be in the running state. In multiuser system, there can be multiple processes which are in the running state.
- **Waiting State:** When a process is in execution, it may request for I/O resources. If the resource is not available, process goes into the waiting state. When the resource is available, the process goes back to ready state.
- **Terminated State:** When the process completes its execution, it goes into the terminated state. In this state the memory occupied by the process is released.

PCB/TCB – Process Control Block / Task Control Block

- Each process is represented as a process control block (PCB) in the operating system. It contains information associated with specific process.
- **Process State:** It indicates current state of a process. Process state can be new, ready, running, waiting and terminated.
- **Process number:** Each process is associated with a unique number which is known process identification number.
- **Program Counter:** It indicates the address of the next instruction to be executed for the process.
- **CPU Registers:** The registers vary in number and type depending on the computer architecture. Register includes accumulators, index registers and general purpose registers plus any condition code information.

Accumulator → Stores results of calculations.

Index Register → Used for addressing and accessing data.

General-Purpose Registers → Used to store temporary data and values.

Condition Code / Status Register → Stores information about the result of operations, such as zero, carry, negative, or overflow.



Memory Management Information: It includes information such as value of base and limit registers, page tables, segment tables, depending on the memory system used by OS.

Base Register → Stores the starting address of the memory allocated to the process.

Limit Register → Stores the size or limit of the memory allocated to the process.

Page Table → Keeps information about the pages of a process and where they are stored in physical memory.

Segment Table → Keeps information about the segments of a process.

Accounting Information: This information includes the amount of CPU used, time limits, account holders, job or process number and so on. It also includes information about listed I/O devices allocated to the process. Each PCB gives information about a particular process for which it is designed.

Process Scheduling -

- Scheduling Queues
- Schedulers
- Context Switch

Scheduling Queues

- All PCBs (Process Control Blocks) are kept in Process Scheduling Queues by the OS. Each processing state has its own queue in the OS, and PCBs from all processes in the same execution state are put in the very same queue. A process's PCB is unlinked from its present queue and then moved to its next state queue when its status changes.
- The following major process scheduling queues are maintained by the Operating System:
- **Job queue** – It contains all of the system's processes.
- **Ready queue** – This queue maintains a list of all processes in the main memory that are ready to run. This queue is always filled with new processes.
- **Device queue** – This queue is made up of processes that are stalled owing to the lack of an I/O device.

Scheduling Queues

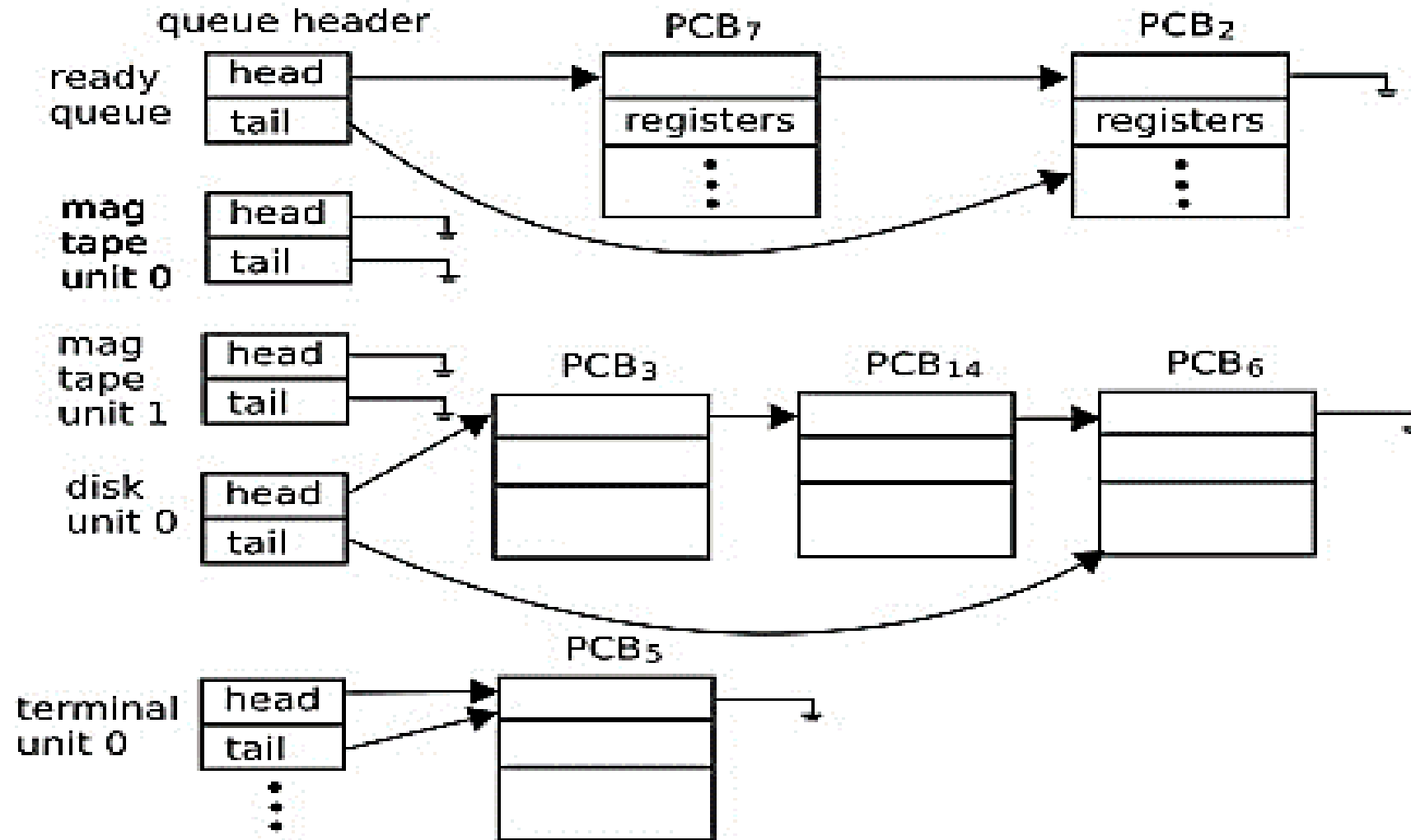


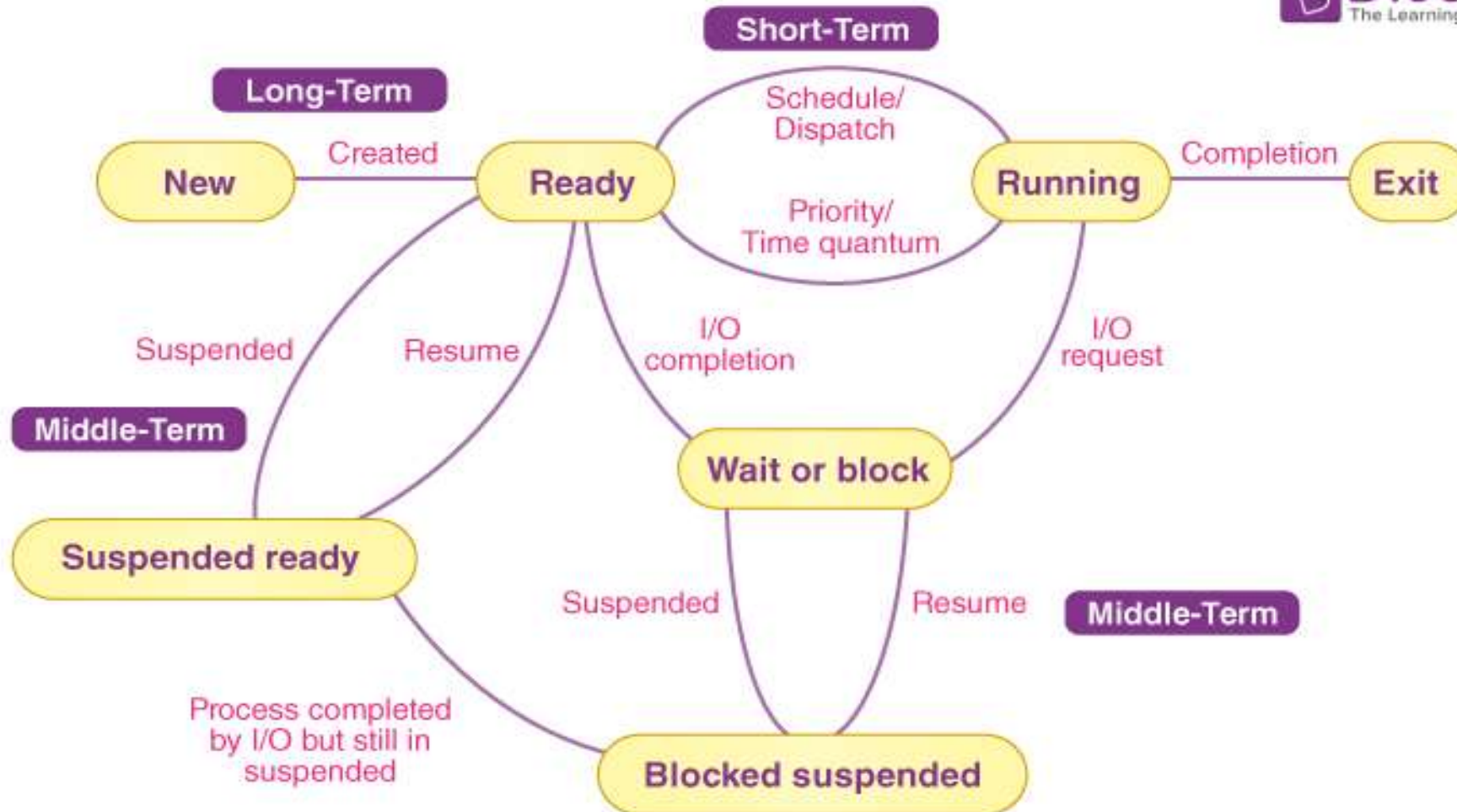
Fig: Ready queue and Device Queue

Schedulers

- A process migrates among the ready queue and various wait queues throughout its lifetime.

OR

- A process moves between the Ready Queue and different Waiting Queues during its execution.
- The role of the CPU scheduler is to select from among the processes that are in the ready queue and allocate a CPU core to one of them.
- The CPU scheduler must select a new process for the CPU frequently.
- Types of Schedulers:
 - Long term Schedulers
 - Mid term Schedulers
 - Short term Schedulers



Long term Scheduler

- Also known as Job Scheduler / Admission Scheduler.
- It selects programs from job pool and loads them into the main memory.
- It controls the degree of multiprogramming.
- The degree of multiprogramming is the number of processes loaded (existing) into the main memory.
- System contains I/O bound processes and CPU bound processes. An I/O bound process spends more time for doing I/O operations whereas CPU bound process spends more time in doing computations with the CPU. So It is the responsibility of long term scheduler to balance the system by loading some I/O bound and some CPU bound processes into the main memory.
- Long term scheduler executes only when a process leaves the system, so it executes less frequently. When long term scheduler selects a process from job pool, the state of process changes from new to ready state.

Short term Scheduler

- It is also known as CPU scheduler.
- This scheduler selects processes that are ready for execution from the ready queue and allocates the CPU to the selected process.
- Frequency of execution of short term scheduler is more than other schedulers.
- When short term scheduler selects a process, the state of process changes from ready to running state.

Medium term scheduler

- It is also known as Swapping Scheduler.
- When a process is in running state, due to some interrupt it is blocked. System swaps out blocked process and store it into a blocked and swapped out process queue.
- When space is available in the main memory, the operating system looks at the list of swapped out but ready processes.
- The medium term scheduler selects one process from that list and loads it into the ready queue. *The job of medium term scheduler is to select a process from swapped out process queue and to load it into the main memory.*
- This scheduler works in close communication with long term scheduler for loading process into the main memory.

Sr. No	Long Term Scheduler	Short Term Scheduler
1	It is job scheduler	It is CPU scheduler
2	It selects processes from job pool and loads them into memory for execution	It selects processes from ready queue which are ready to execute and allocates CPU to one of them
3	Access job pool and ready queue	Access ready queue and CPU
4	It executes much less frequently. It executes when memory has space to accommodate new process.	It executes frequently. It executes when CPU is available for allocation
5	Speed is less than short term scheduler	Speed is fast
6	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming
7	It chooses a good process that is a mix-up of input/output bound and CPU bound.	It chooses a new process for a processor quite frequently.

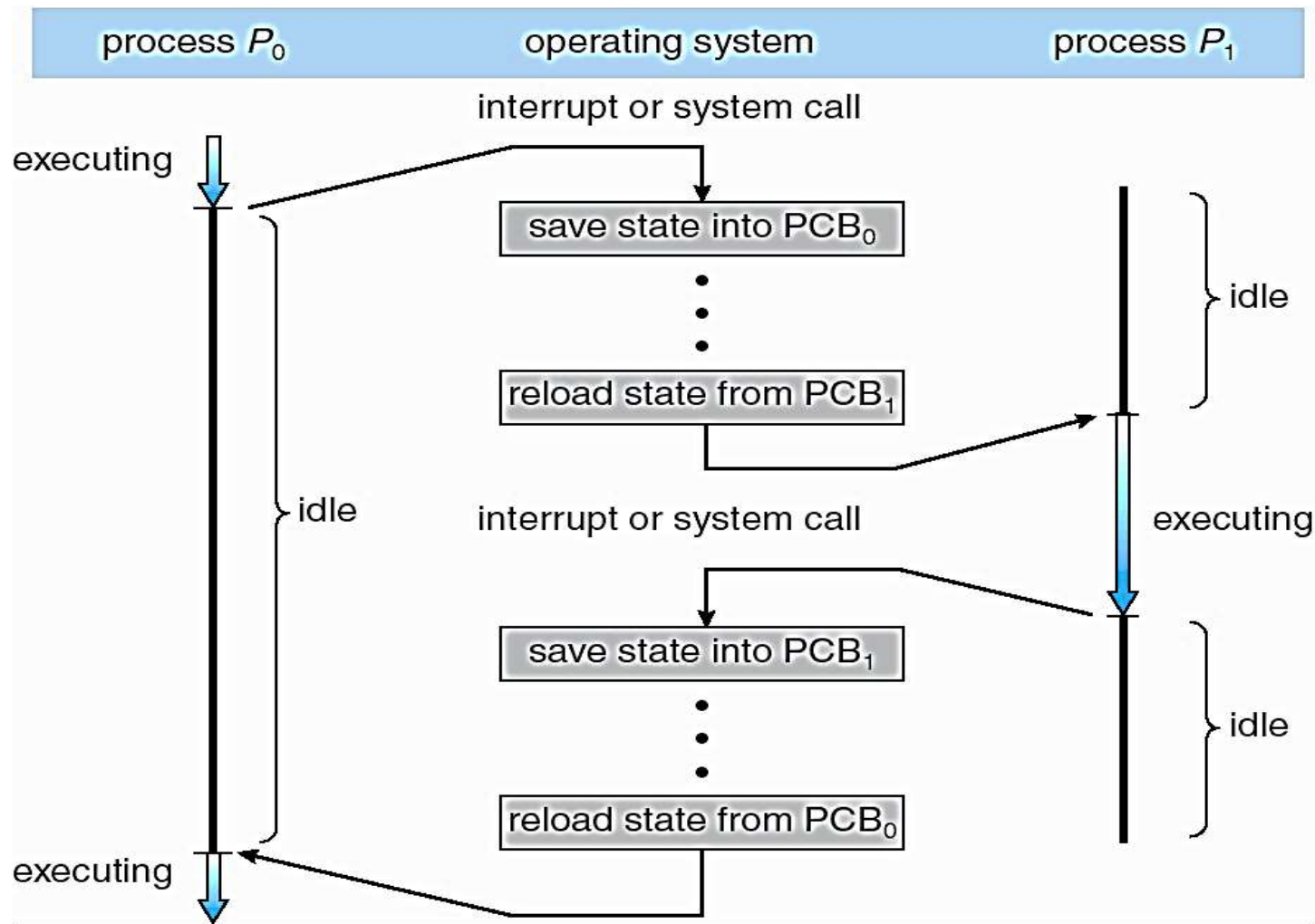
Context Switch

- A CPU switch from process to process is referred as context switch.
- A context switch is a **mechanism** that store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time.

Context Switch

- When the scheduler switches the CPU from one process to another process, the context switch saves the contents of all process registers for the process being removed from the CPU, in its process control block. Context switch includes two operations such as state save and state restore. State save operation stores the current information of running process into its PCB. State restore operation restores the information of process to be executed from its PCB.
- Switching the CPU from one process to another process requires performing state save operation for the currently executing process (blocked) and a state restore operation for the process ready for execution. This task is known as context switch.

CPU Switch From Process to Process – Context Switch



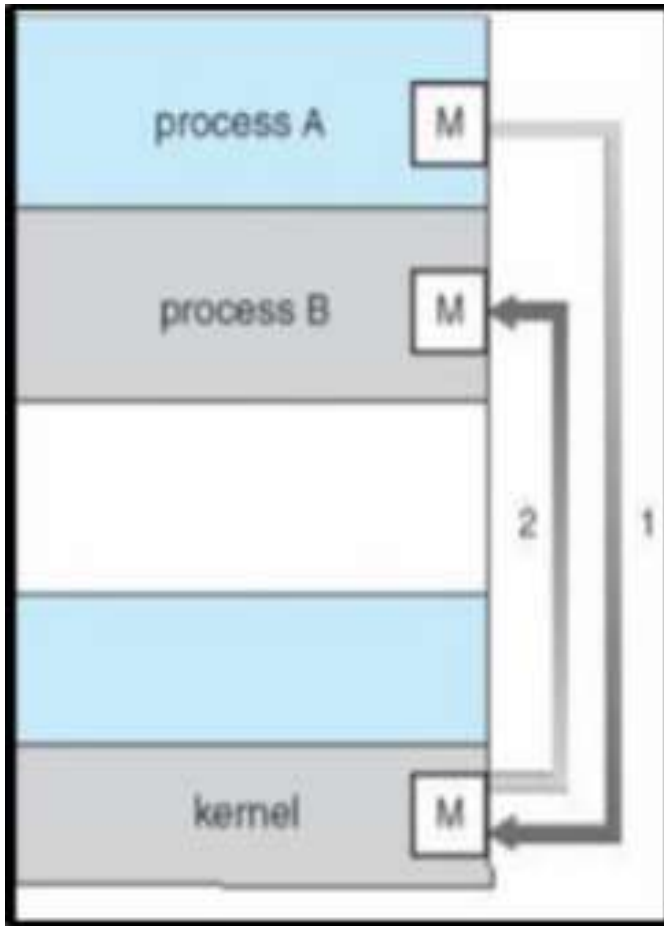
Interprocess Communication



- Interprocess communication is the mechanism provided by the operating system that allows processes to communicate with each other.
- Cooperating processes require an Inter-process communication (IPC) mechanism that will allow them to exchange data and information.
- It helps processes work together efficiently and safely in an operating system.
- There are two methods of IPC, shared memory and message passing. An operating system can implement both methods of communication.

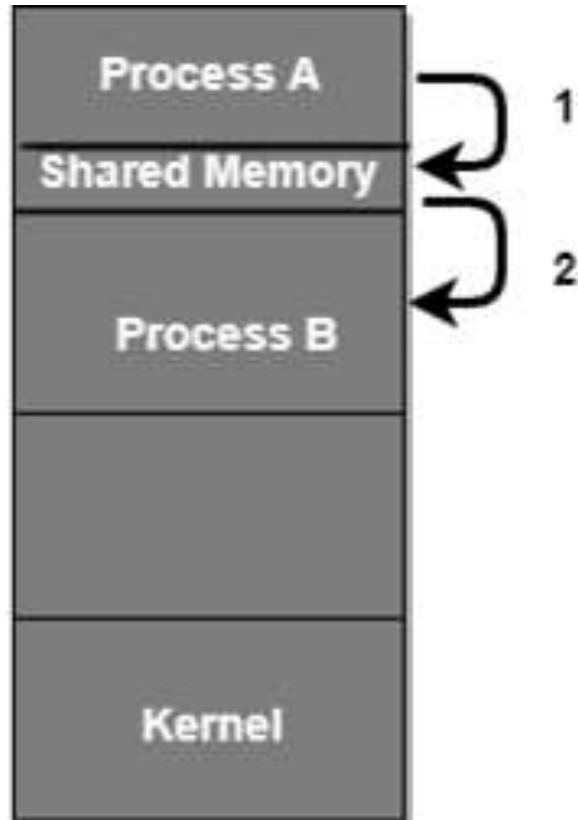
Example: A simple example of IPC is a bank ATM system, where one process reads the card and PIN, another checks the account balance, and a third dispenses cash. These processes communicate and coordinate to complete the transaction correctly.

Message Sharing/ Passing



- In this model, communication takes place by exchanging messages between cooperating processes.
- It allows processes to communicate and synchronize their action without sharing the same address space.
- Message Passing is very useful in a distributed system, where different processes may be running on different computers connected through a network.
- Communication requires sending and receiving messages through the kernel.
- The processes that want to communicate with each other must have a communication link between them. Between each pair of processes exactly one communication link.

Shared Memory



- In this a region of the memory residing in an address space of a process creating a shared memory segment can be accessed by all processes who want to communicate with other processes.
- All the processes using the shared memory segment should attach to the address space of the shared memory.
- All the processes can exchange information by reading and/or writing data in shared memory segment.

- All processes can exchange information by reading and/or writing data to the shared memory segment.
- The format of the data and the location are determined by the processes that want to communicate with each other.
- These processes are not under the control of the Kernel.
- The Processes are also responsible for ensuring that they are not writing to the same location at the same time.
- Once a shared memory segment is established, all accesses to the shared memory segment are treated as normal memory accesses and without kernel assistance.
- It is particularly useful in a distributed environment when communication process may reside on a different computer connected by a network.

3.4 Threads

3.4.1 Benefits

3.4.2 Users and Kernel Threads

3.4.3 Multithreading Models

3.4.3.1 Many to One

3.4.3.2 One to One

3.4.3.3 Many to Many

Threads

- A **thread** is a small unit of execution within a process. It represents a **flow of instructions** that are being executed. A thread has three important things:
- **Program Counter (PC):** Keeps track of **which instruction should be executed next**.
- **System Registers:** Store the thread's **current data and working variables**.
- **Stack:** Stores the **function calls and execution history** of the thread.
- A thread is also called a **lightweight process**.
- Each thread belongs to exactly one process and no thread can exist outside a process.

Benefits - Threads

- Threads minimize the context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- It is more economical to create and context switch threads.
- Threads share code, data, and operating system resources within the same process.
- Threads can use multiple processor cores at the same time, which improves the performance and efficiency of a computer system.

Difference between Process and Thread

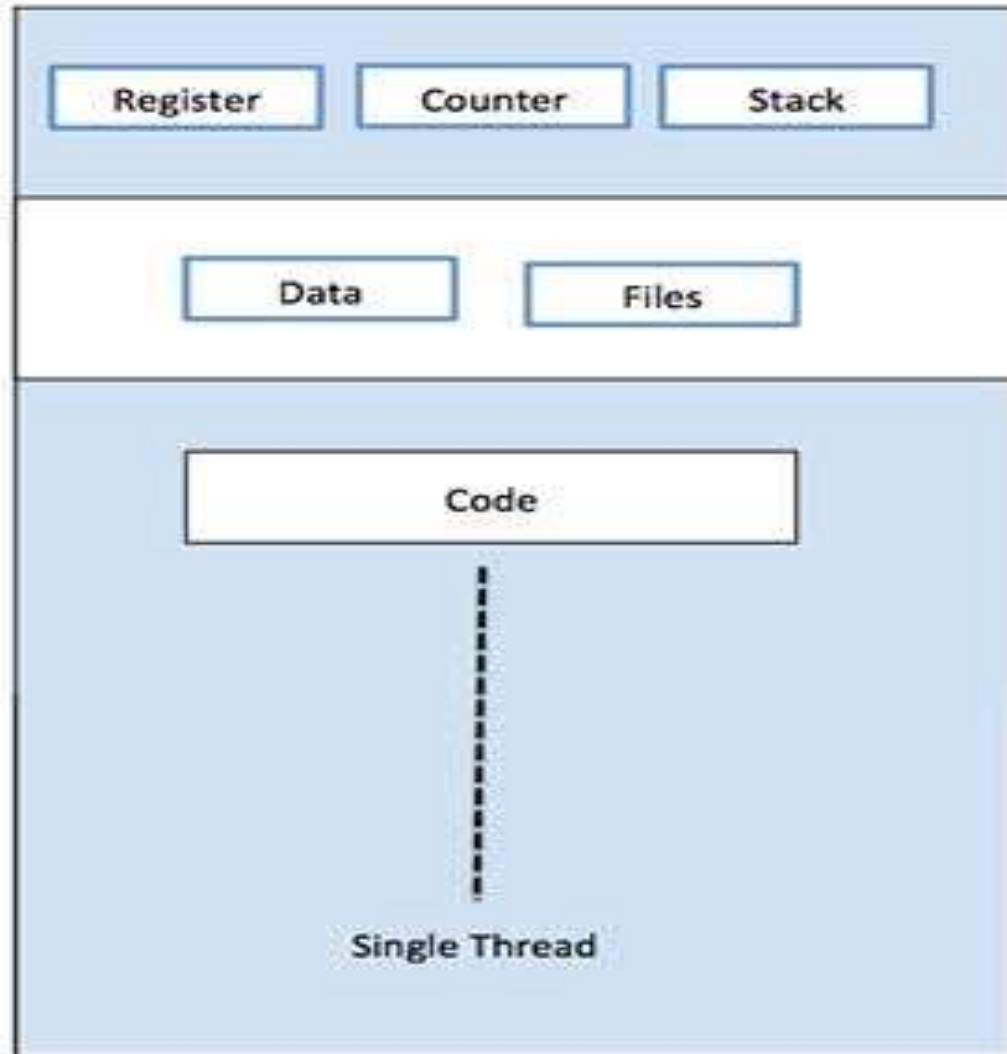
Process

- Process is heavy weight or resource intensive.
- Process switching needs interaction with operating system.
- If one process is blocked, then no other process can execute until the first process is unblocked.
- Multiple processes without using threads use more resources.
- In multiple processes each process operates independently of the others.
- Takes more time to create & terminate.
- Program in execution
- Context switching is slow
- Does not share data
- Less efficient communication

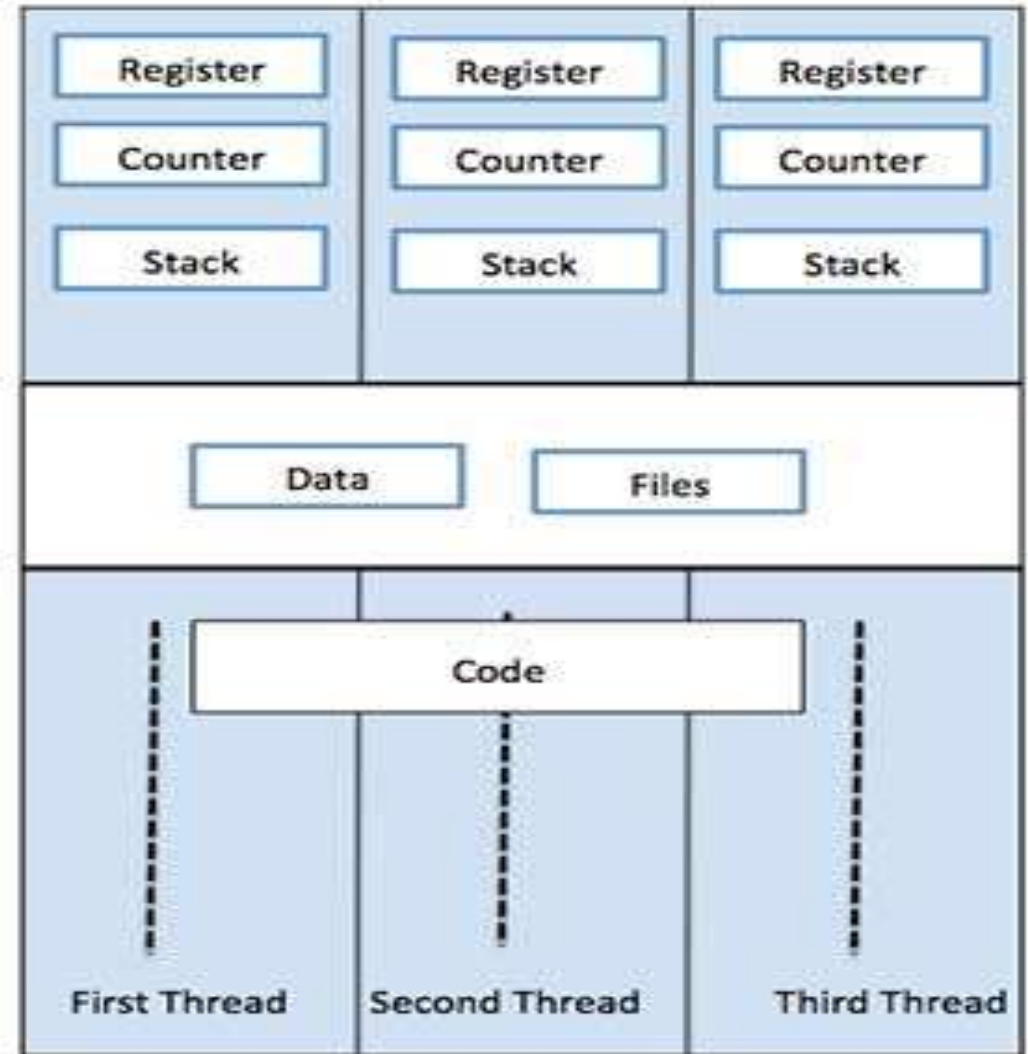
Thread

- Thread is light weight, taking lesser resources than a process.
- Thread switching does not need to interact with operating system.
- While one thread is blocked and waiting, a second thread in the same task can run.
- Multiple threaded processes use fewer resources.
- One thread can read, write or change another thread's data.
- Takes less time to create & terminate.
- Part of a process
- Context switching is fast
- Shares data with other threads
- More efficient communication

Threads



Single Process P with single thread

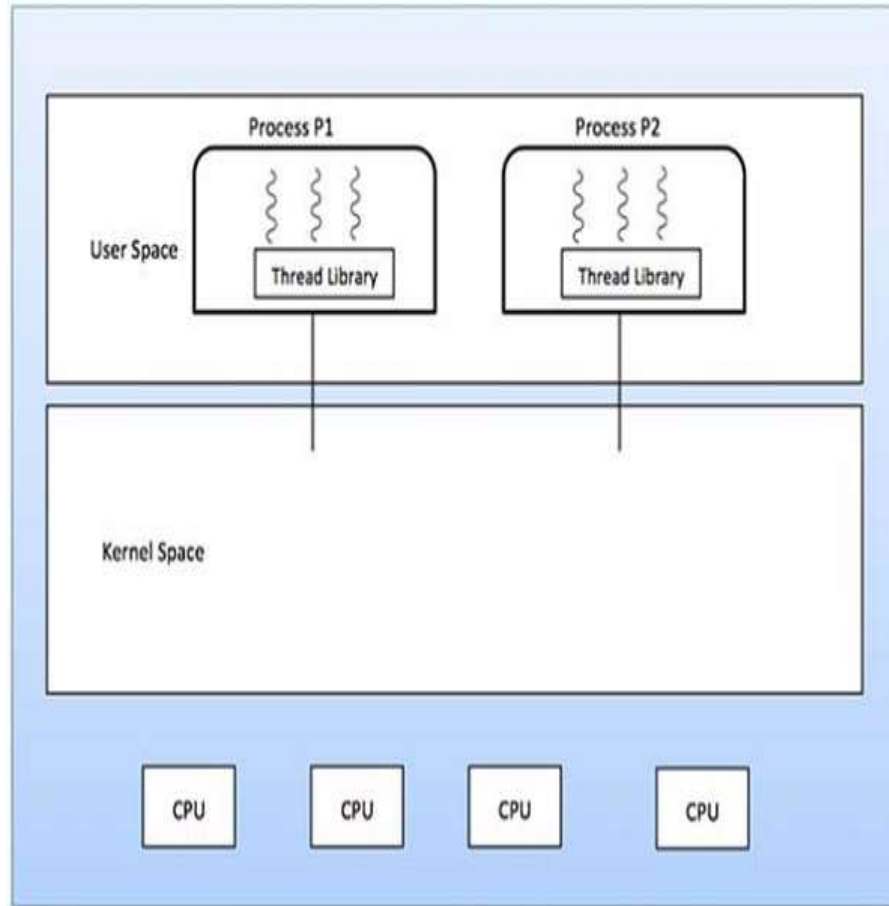


Single Process P with three threads

Types of Thread

1. **User Level Threads** – User managed threads.
2. **Kernel Level Threads** – Operating System managed threads acting on kernel, an operating system core.

User Level Threads



- The thread management kernel is not aware of the existence of threads.
- The thread library contains code for creating and destroying threads, for passing message and data between threads, for scheduling thread execution and for saving and restoring thread contexts. The application starts with a single thread.

Advantages

- Thread switching does not require Kernel mode privileges.
- User level thread can run on any operating system.
- The application can decide how its user-level threads are scheduled according to its own requirements.
- User level threads are fast to create and manage

Disadvantages

- In a operating system, most system calls are blocking.
- Multithreaded application cannot take advantage of multiprocessing.

Kernel Level Threads

- Thread management is done by the Kernel.
- Kernel threads are supported directly by the operating system.
- There is no thread management code in the application area.
- All of the threads within an application are supported within a single process.
- Scheduling by the Kernel is done on a thread basis.
- Kernel can schedule multiple threads belonging to the same process on multiple CPU cores at the same time.
- Any application can be programmed to be multithreaded.

Advantages

- If one thread in a process is blocked, the Kernel can schedule another thread of the same process.
- Kernel routines themselves can be multithreaded.
- Allows true parallel execution of threads

Disadvantages

- Kernel threads are generally slower to create and manage than the user threads.
- More complex to implement than user-level threads.
- Context switching is slower compared to user-level.

Difference between User-Level & Kernel-Level Thread

S.N.	User-Level Threads	Kernel-Level Thread
1	User-level threads are faster to create and manage.	Kernel-level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User-level thread is generic and can run on any operating system.	Kernel-level thread is specific to the operating system.
4	Multi-threaded applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

Multithreading Models

- Some operating system provide a combined user level thread and Kernel level thread facility.
- Multithreading improves system performance and responsiveness by allowing multiple threads to share CPU, memory and I/O resources of a single process.
- Example: In a browser, each tab can be a thread. In MS Word, one thread formats text while another processes inputs.
- Multithreading models are three types
 - Many to many relationship.
 - Many to one relationship.
 - One to one relationship

Many to One

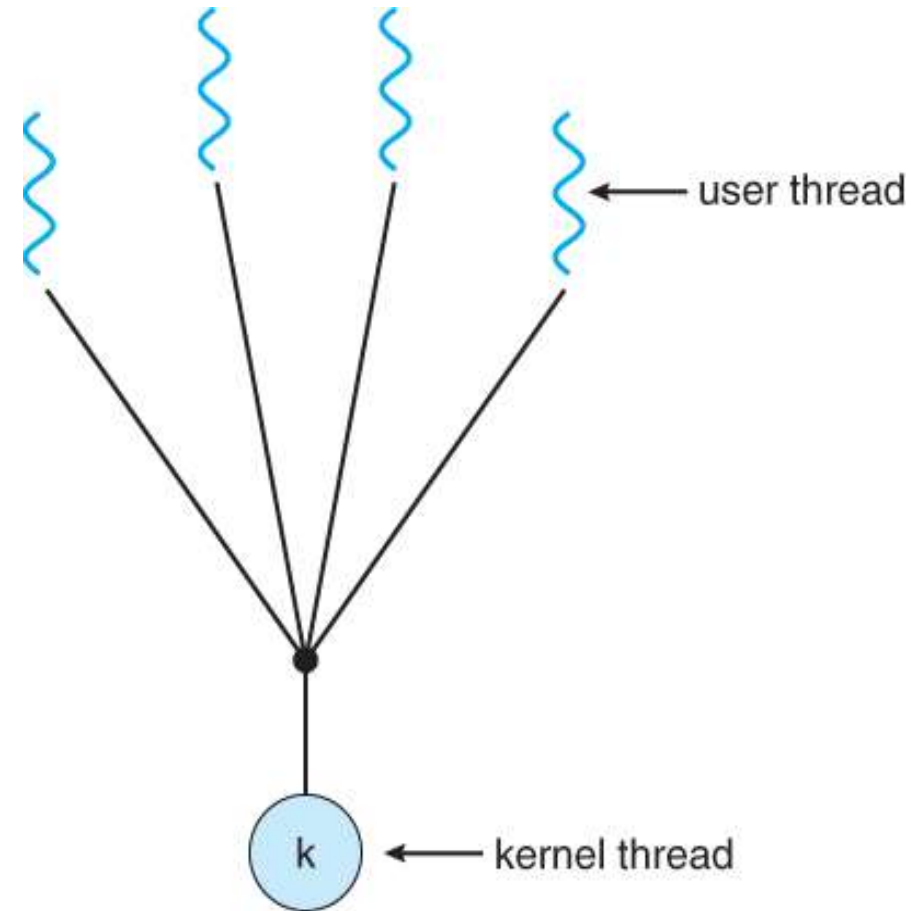
- This model maps many user level threads to one kernel level thread. Thread management is done by thread library in user space.

Advantages:-

- It is an efficient model as threads are managed by thread library in user space

Disadvantages:-

- Only one thread can access the kernel at a time, so multiple threads are unable to run in parallel on microprocessor.
- If a thread makes a blocking system call then the entire process will be block.



One-to-One

- It maps each user thread to a kernel thread. Even a thread makes a blocking call; other thread can run with the kernel thread.

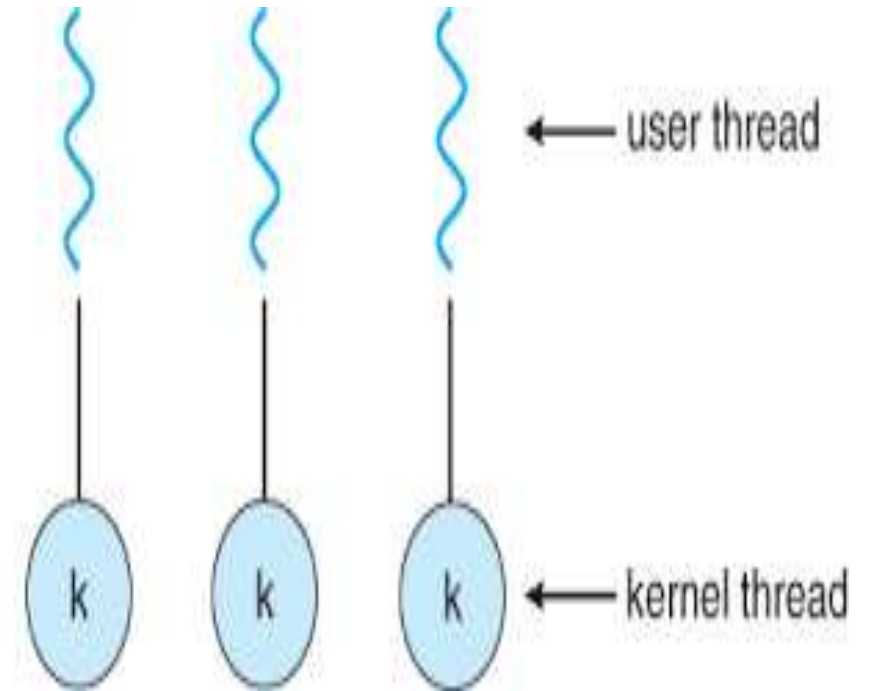
- **Advantages:-**

It allows multiple threads to run in parallel on multiprocessors.

- **Disadvantages:-**

Creating a user thread requires creating the corresponding kernel thread.

Creating kernel thread may affect the performance of an application.

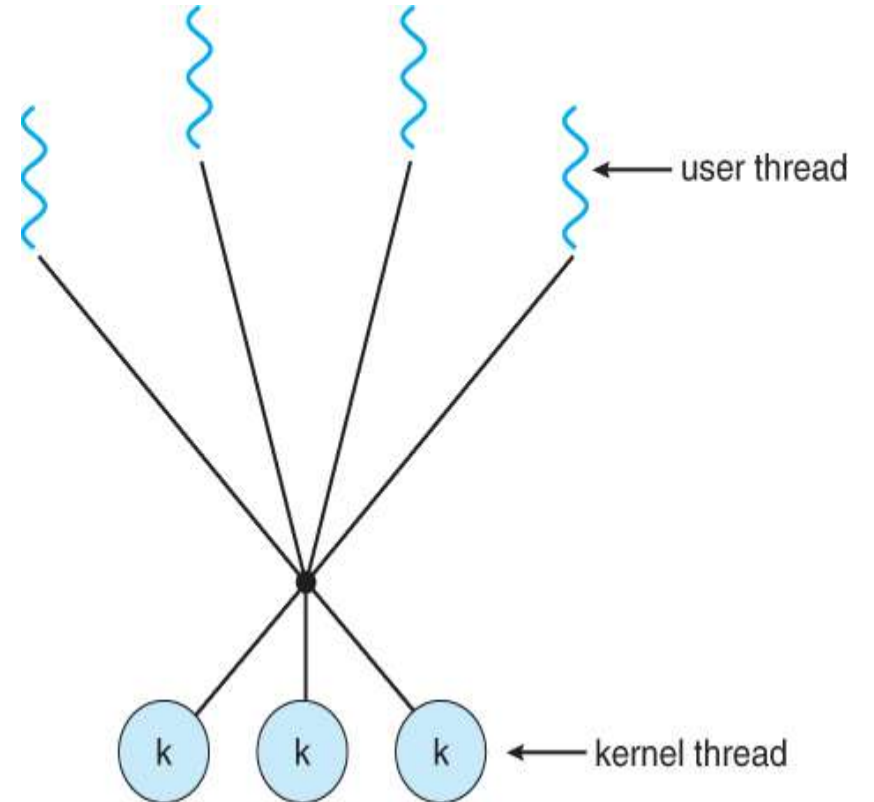


Many to Many

- This model maps many user level threads to a smaller or equal number of kernel threads. Number of kernel threads may be specific to either a particular application or particular machine.

Advantages:-

- Developer can create as many user threads as necessary. Threads can run in parallel on a multiprocessor.
- When a thread performs a blocking system call, the kernel can schedule another thread for execution.



Execute Process Commands

- Ps
- Wait
- Sleep
- Exit
- Kill